

Linear Regression: The 7-Step Workflow

A start-to-end process, with the checkpoint that tells you the model is fine at each stage.

1 Understand the data

Load the dataset and inspect it before touching a model. Check the shape, look for missing values, review feature ranges, and see which features relate to the target.

```
df.shape
df.isnull().sum()
df.describe()
df.corr()
```

Fine check: no unexpected nulls, target is continuous, some features correlate with the target.

2 Define X and y, then split

Separate the features (X) from the target (y). Hold out a test set that the model never sees during training. It is your stand-in for real, unseen data.

```
X = df.drop('target', axis=1)
y = df['target']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

Fine check: test set is held out and stays untouched from here on.

3 Preprocess inside a pipeline

Scale, encode, or impute as needed. Wrapping steps in a Pipeline guarantees preprocessing is learned from training data only, then applied to test. This prevents data leakage.

```
model = Pipeline([
    ('scaler', StandardScaler()),
    ('linear_reg', LinearRegression())
])
```

Fine check: the scaler is fit on train and applied to test, never fit on the full data.

4 Train

Fit the pipeline on the training data. The scaler learns its parameters and the regression learns its coefficients in one call.

```
model.fit(X_train, y_train)
```

Fine check: it runs cleanly and the input features make sense for the target.

5 Predict on both sets

Generate predictions for the training set and the test set. You need both, because comparing them is how you diagnose the model in the next step.

```
train_pred = model.predict(X_train)
test_pred = model.predict(X_test)
```

Fine check: predictions are produced for train and test separately.

6 Evaluate

Compute metrics (MAE, RMSE, R2) on both sets and place them side by side. The test score answers 'how good is the model?' The train-vs-test gap answers 'can I trust it to generalize?' RMSE is in target units; R2 is the fraction of variance explained. If RMSE is much larger than MAE, a few big errors exist.

```
test_metrics = calculate_regression_metrics(y_test, test_pred)
train_metrics = calculate_regression_metrics(y_train, train_pred)
# then compare them side by side
```

Fine check: you have a test score for quality AND a train-vs-test comparison for trust.

7 Diagnose and iterate

Interpret step 6 through the bias-variance lens. Weak train score with test close to it means underfitting (high bias): reach for a more expressive model. Strong train score with a much weaker test score means overfitting (high variance): simplify, regularize, or get more data. Strong and close means you are done. Confirm any single split with cross-validation so the result is not a fluke.

```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(model, X, y, cv=5, scoring='r2')
print(scores.mean(), scores.std())
```

Fine check: train and test are close AND the score meets the goal AND residuals are random.

How to tell if the model is fine

Pattern	Meaning	Action
Train $\approx$ Test, both strong	Healthy model	Ship it
Train $\approx$ Test, both weak	Underfitting (high bias)	Stronger model, more features
Train $\blacksquare$ Test	Overfitting (high variance)	Regularize, simplify, more data

In one paragraph: Explore the data, hold out a test set, preprocess without leakage, train, then predict on both train and test. Compare the two to diagnose bias vs variance, check whether the score meets your goal, look at residuals for hidden patterns, and confirm with cross-validation that the result is stable. A model is fine when it generalizes (train $\approx$ test), meets the use-case bar, and has random residuals.